



Module 12: Algorithmic Strategies

CPE010 DATA STRUCTURES AND ALGORITHMS

Review and Introduction to Various Algorithmic Design Strategies

Intended Learning Outcomes

After this module, the student should be able to:

- Measure the runtime of algorithms using theoretical analysis
- Analyze the results of runtime performance by comparing experimental and theoretical analysis

General Concepts

- Algorithm **strategy**
 - Approach to solving a problem
 - May combine several approaches
- Algorithm **structure**
 - Iterative -> execute action in **loop**
 - Recursive -> re-apply action to **subproblems**
- **Problem type**
 - Satisfying -> find **any** satisfactory solution
 - Optimization -> find **best** solutions (vs. cost metric)

Some Algorithmic Strategies

Covered in this module:

- Recursion
- Brute Force
- Backtracking
- Divide-and-Conquer
- Dynamic Programming
- Greedy

Other strategies (for further reading):

- Branch and Bound Algorithms
- Heuristic Algorithms

Recursive Algorithm

Based on re-applying algorithm to subproblem.

The approach:

1. Solve base case(s) directly.
2. Recurs with a simpler subproblem.
3. Optional: Convert solutions to subproblems.

Example 1 - Recursive Algorithm

Algorithm for computing the k-th power of 2.

Algorithm `Power_of_2`(natural number k)

Input: k, a natural number

Output: k-th power of 2

Algorithm:

If k = 0, **then** return 1;

Else return `2*Power_of_2(k-1)` .

Example 2 - Recursive Algorithm

Recursive algorithm for sequential search.

Algorithm 3 **SeqSearch(L, i, j, x)**

Input: L is an array, i and j are positive integers, $i \leq j$, and x is the key to be searched for in L.

Output: If x is in L between indexes i and j, then output its index, else output 0.

Algorithm:

```
if  $i \leq j$  , then
{
    if  $L(i) = x$ , then return i ;
    else return SeqSearch(L, i+1, j, x)
}
else return 0.
```

Brute Force Algorithm

Based on trying **all possible solutions**. Generally the **most expensive** approach.

Approach:

1. Generate and evaluate possible solutions until:
 - Satisfactory solution is found.
 - Best solution is found (if it can be determined).
 - All possible solutions found:
 - Return best solution.
 - Return failure if no satisfactory solution.

Example 3 - Brute Force Algorithm

Algorithm for **Brute Force String Matching**.

Pattern: A string of m characters to search for.

Text: A longer string of n characters to search in.

Problem: Find a substring in the text that matches the pattern.

Algorithm:

1. Align pattern at the beginning of text.
2. Move from left to right, compare each character of pattern to the corresponding character in text, until:
 - a. All characters are found to match (successful search); or
 - b. A mismatch is detected.
3. While pattern is not found and the text is not yet exhausted, realign pattern one position to the right and repeat step 2.

Example 3 (cont.)

Create a function that will implement Brute Force String Matching.

```
BruteForceStringMatch(T[0...n-1], P[0...m-1])  
For i <- 0 to n - m do  
  j <- 0  
  While j < m and P[j] = T[i+j] do  
    j <- j+1  
  If j = m return i  
Return -1
```

T[0...n-1] represents **text**
P[0...m-1] represents **pattern**

Test it on the following:

- **Pattern:** happy
- **Text:** It is never too late to have a happy childhood.

Backtracking Algorithm

Based on **depth-first recursive search**. Sometimes referred to as “search tree”

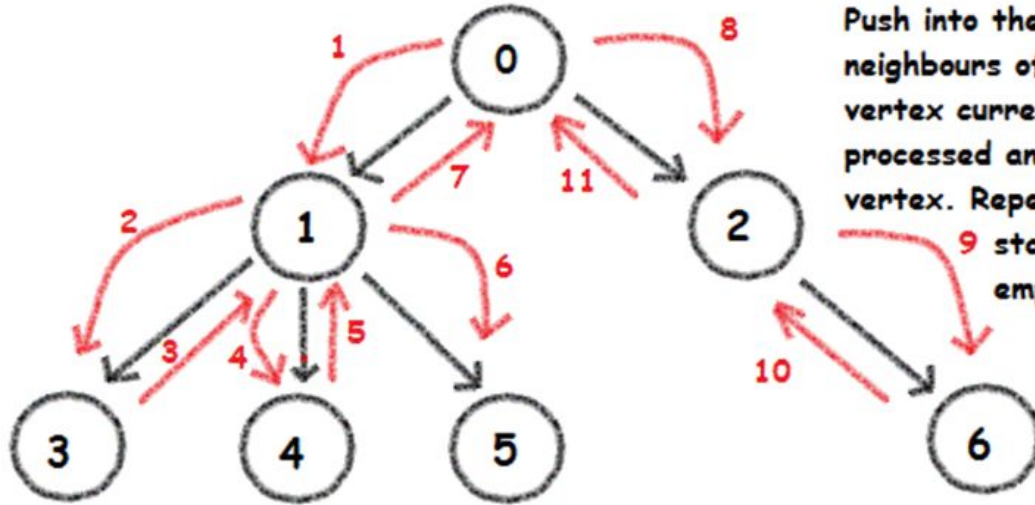
Approach:

1. Tests whether solution has been found
2. If found solution, return it
3. Else for each choice that can be made:
 - Make that choice
 - Recur
 - If recursion returns a solution, return it
4. If no choices remain, return failure

Example 4 - Backtracking

Visualization for Depth First Search that shows backtracking.

Red arrows indicate the order of search.



Push into the stack the neighbours of the vertex currently being processed and Pop the vertex. Repeat until 9 stack is not empty.

Vertex	Stack
	0
0	1, 2
1	3, 4, 5, 2
3	4, 5, 2
4	5, 2
5	2
2	6
6	

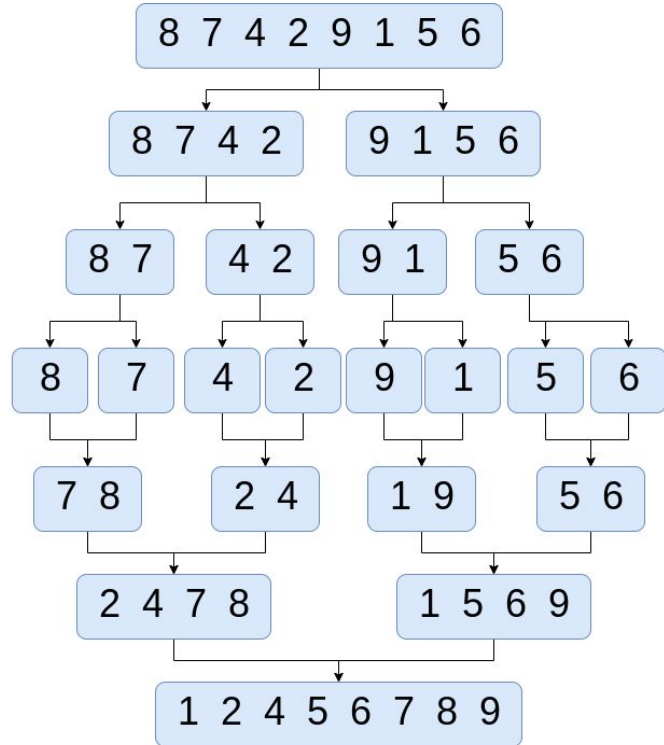
Divide-and-Conquer

Based on **dividing problems into subproblems**. Usually contains two or more recursive calls.

Approach:

1. Divide problem into smaller subproblems
 - Subproblems must be of same type
 - Subproblems do not need to overlap
2. Solve each subproblem recursively
3. Combine solutions to solve original problem

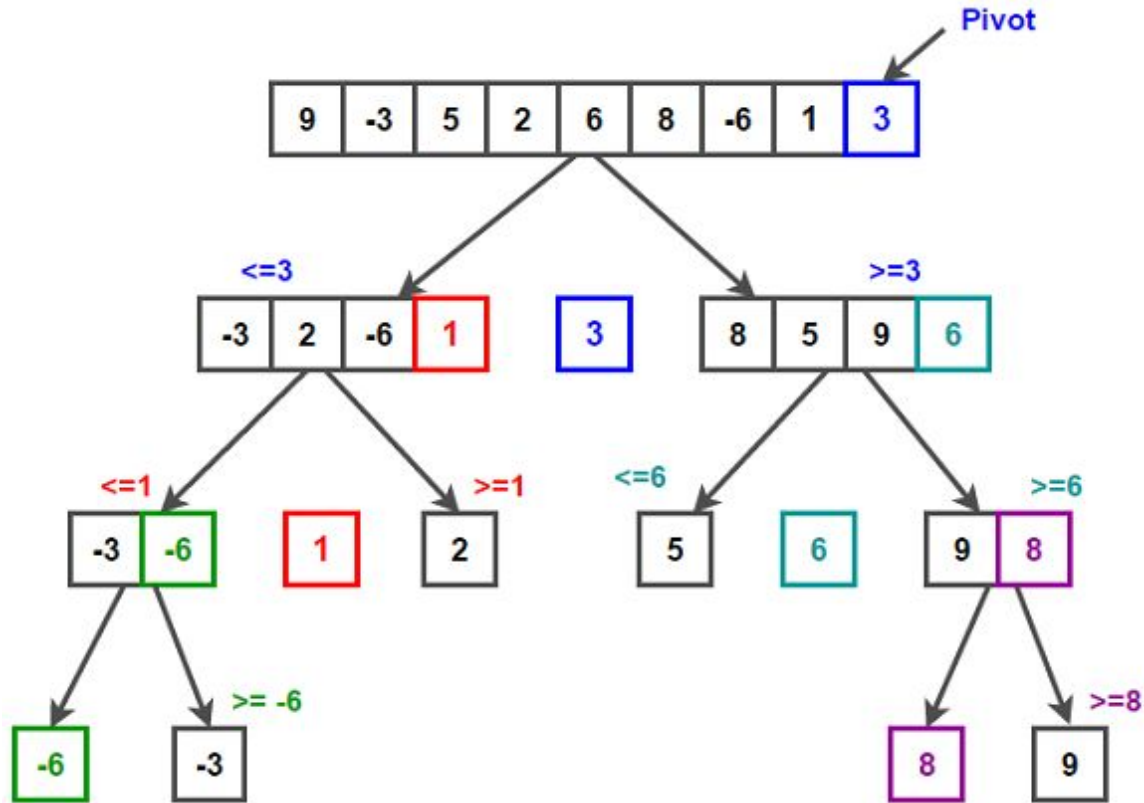
Example 5 - Divide-and-Conquer



What kind of sorting algorithm is shown here?

How does it show divide-and-conquer?

Example 6 - Divide-and-Conquer



What kind of sorting algorithm is shown here?

How does it show divide-and-conquer?

Greedy Algorithm

Based on trying best current (**local**) choice. **Avoid backtracking, exponential time $O(2^n)$**

Approach:

1. At each step of algorithm.
2. Choose best local solution.

Hope local optimum lead to **global** optimum.

Greedy Property

Consists of two properties:

1. **Greedy-Choice Property:** We can arrive at global optimal solution through the local optimal choice.
2. **Optimal Substructure:** Optimal global solution contains the optimal solutions of all the subproblems.

Example:

Providing someone's "change" based on available coins such as: 100 cents, 25 cents, 10 cents, 15 cents and 1 cent.

Advantage & Disadvantages

Advantages

- ❑ Easier to implement.
- ❑ Require much less computing resources.
- ❑ Much faster to execute.
- ❑ Used to solve optimization problems.

Disadvantages

- ❑ May not always reach global optimal solution.
- ❑

Example 7 - Greedy Algorithm

Problem:

- Make a change of a given amount using the smallest possible number of coins.

Solution:

- The coin is selected using greedy criterion: at each stage increases the total amount of change constructed as much as possible.
- To assure feasibility: the change given exactly equals the desired amount of the solution.

Make change for n units using the least possible number of coins.

```
Algorithm: make_change(n):  
const C <- {100, 25, 15, 10, 1}  
S <- {} //empty solution set  
Sum <- 0 sum of items in solution set
```

```
    WHILE sum not = n  
        x = largest item in set C such that sum + x < or = n  
        IF no such item THEN  
            RETURN "No Solution"  
        S <- S{value of x}  
        Remove x from C  
        sum <- sum + x  
RETURN S
```

Create code for the implementation of this greedy algorithm to solve the given problem.

Dynamic Programming Algorithm

Based on **remembering past results**. Generally applied to **optimization** problems.

Approach:

1. Divide problems into smaller subproblems.
 - Subproblems must be of same type.
 - Subproblem must **overlap**.
2. Solve each subproblem recursively.
 - May simply look up solution.
3. Combine solutions into one to solve original problem.
4. Store solution to problem.

Application of Dynamic Programming

Top-Down

Problem is broken down.
Follows the **memoization** technique: consisting of **recursion** and **caching**.

Memoize **is not** memorize.

If the problem is already solved;
a saved value is returned. Else,
the value of the function is
calculated for the first time;
then saved.

Bottom-Up

Uses the **tabulation** technique;
without the use of recursion.

Instead of recursion, it uses
iteration.

This method avoids **stack overflow** or **recursive procedure overhead**.

Example 8 - Fibonacci Series

Fibonacci series is a sequence of numbers in such a way that each number is the sum of the two preceding ones, starting from 0 and 1.

$$F(n) = F(n-1) + F(n-2)$$

In C++, this can be done simply through the function:

```
int fibonacci(int n){  
    if(n <= 1){  
        return n;  
    } else {  
        return (fibonacci(n-1) + fibonacci(n-2));  
    }  
}
```

Example 8 - Memoization

```
int memArr[1000]; this will be your memoization dictionary
int fibTD(int n, int *memo) {
    { if(memo[n] != 0) {
        return memo[n];
    } } If already computed, no further computations;
        this reduces the number of work needed.
    { if(n <= 1) {
        return n;
    } } Compute for the base case
    else {
        memo[n] = fibonacci(n-1) + fibonacci(n-2);
        return memo[n];
    }
}
```

For this new computation, store in the dictionary, this facilitates the prevention of future computation for the same term.

Example 8 - Memoization

Your main function would look something like this:

```
int main() {  
    for(int i = 0; i < 5; i++) {  
        std::cout << fibTD(i, memArr) << " ";  
    }  
  
    return 0;  
}
```

Example 8 - Tabulation Technique

```
int fibBU(int n){  
    int *table = new int[n+1]; Build the table.  
    {  
        table[0] = 0;  
        table[1] = 1;  
    } Building solution to subproblems from the bottom up.  
  
    {  
        for(int i = 2; i <= n; i++){  
            table[i] = table[i-1] + table[i-2];  
        }  
    } No more recomputation of  
        computed values; less  
        auxiliary space compared  
        to recursion.  
  
    return table[n];  
}
```

Key Differences: Top-Down vs Bottom-Up

Top-Down Approach	Bottom-Up Approach
Uses memorization technique	Uses tabulation technique
Recursive nature	Iterative nature
Structured programming languages such as COBOL, Fortran, C, and others mostly use this technique	Object-oriented programming languages such as C++, C#, and Python mostly use this technique
This approach uses decomposition to formulate a solution	This approach uses composition to develop a solution
A lookup table is maintained and checked before computation of any subproblem.	The solution is built from a bottom-most case using iteration.

Concept Check:

Find the factorial of a number N using Dynamic Programming techniques



Thanks for participating!

Submit your questions through the discussion
Q&A found on module 12.